অধ্যায় ২৬

২৬

Advanced Database Integration, ORM, Query Optimization এবং Scalable Data Architecture

---

শেখার লক্ষ্য

এই অধ্যায় শেষ করার পর আপনি পারবেন:

- Relational Model-এ তথ্য সাজাতে এবং Normalization দিয়ে পুনরাবৃত্তি ও অসামঞ্জস্য দূর করতে।
- Primary Key, Foreign Key, UNIQUE, CHECK ও NOT NULL Constraint দিয়ে Database-কেই নিয়মের পাহারাদার বানাতে।
- Transaction-এর ACID বৈশিষ্ট্য বুঝতে, এবং Rollback ও Savepoint ব্যবহার করতে।
- Isolation, Lock, Lost Update ও তার প্রতিরোধ বুঝতে।
- Index, `EXPLAIN QUERY PLAN` ও N+1 সমস্যা দিয়ে Query-র গতি বিশ্লেষণ করতে।
- সংস্করণভিত্তিক Migration লিখতে এবং চালানো Migration-এর পরিবর্তন ধরতে।
- Backup তৈরি ও যাচাই করতে।
- ORM (Prisma) ও Query Builder-এর সুবিধা ও সীমা বিচার করতে, এবং SQLite থেকে PostgreSQL-এ যাওয়ার পার্থক্য বুঝতে।
- একটি সম্পূর্ণ পরীক্ষিত **TravelData Pro** Loyalty Ledger তৈরি করতে।

পূর্বশর্ত

- অধ্যায় ২০: SQL Injection ও Parameterized Query, `node:sqlite`।
- অধ্যায় ২১: Repository Pattern।
- অধ্যায় ২২: Property-based Test, Integration Test।
- অধ্যায় ২৫: Repository-র `async` চুক্তি, Idempotency-Key, Optimistic Concurrency।

এই অধ্যায় কেন গুরুত্বপূর্ণ

একটি ভ্রমণ সংস্থার Loyalty Program: প্রতিটি Tour-এ গ্রাহক Point পান, Point দিয়ে হোটেলের রাত বা ছাড় নেন, পরিবারের সদস্যকে Point দেন। এখানে টাকার মতোই কঠোর নিয়ম: কোনো Point হাওয়া থেকে আসবে না, কোনো Point হারাবে না, কারও Balance ঋণাত্মক হবে না, এবং একই Tour-এর Point দুবার যোগ হবে না—Network বিভ্রাট, দুজন কর্মীর একসঙ্গে কাজ বা Server-এর হঠাৎ বন্ধ হওয়া সত্ত্বেও।

এই নিয়মগুলো শুধু JavaScript-এ লিখলে যথেষ্ট নয়। একটি Script Database-এ সরাসরি লিখতে পারে, দুটি Server একসঙ্গে একই Balance পড়তে পারে, একটি Process দুটি লেখার মাঝখানে বন্ধ হতে পারে। **Database** এই নিশ্চয়তাগুলো দেওয়ার জন্যই তৈরি: Constraint, Transaction, Lock। এই অধ্যায় শেখায় সেগুলো কীভাবে কাজে লাগাতে হয়—এবং কীভাবে Query-কে লাখো সারিতেও দ্রুত রাখতে হয়।

২৬.১ আগে যা শিখেছেন, এখন যা নতুন

আগে শেখা	এই অধ্যায়ে নতুন
Parameterized Query (অধ্যায় ২০)	Schema-র নকশা, Normalization, Constraint
Repository Pattern (অধ্যায় ২১, ২৫)	Transaction, Rollback, Savepoint
Optimistic Concurrency (অধ্যায় ২৫)	Lock, Isolation, Lost Update-এর কারণ
Idempotency-Key (অধ্যায় ২৫)	UNIQUE Constraint দিয়ে Database-স্তরের Idempotency
Big-O ও Profiling (অধ্যায় ১৯, ২৩)	Index, Query Plan, N+1
—	Migration, Backup, ORM, PostgreSQL-এর পার্থক্য

> **নোট:** v30-এর এই অধ্যায় PostgreSQL ও Prisma-কেন্দ্রিক। এই বইয়ের QA-যন্ত্রে কোনো PostgreSQL Server বা Prisma ছিল না, এবং কোনো Package Install করা হয়নি। তাই সব চালানো Code Node.js-এর অন্তর্নির্মিত `node:sqlite` (SQLite) দিয়ে, যেখানে Constraint, Transaction, Lock, Index ও Query Plan—সবই আসল Database-এর আচরণ। PostgreSQL ও Prisma-র Code যেখানে দেখানো হয়েছে, সেখানে স্পষ্ট লেখা আছে যে তা চালানো হয়নি। দুই Database-এর পার্থক্য অনুচ্ছেদ ২৬.৯-এ।

২৬.২ Relational Model এবং Normalization

Relational Database-এ তথ্য থাকে Table-এ: প্রতিটি সারি একটি জিনিস, প্রতিটি Column একটি বৈশিষ্ট্য, এবং **Primary Key** প্রতিটি সারিকে আলাদা করে চেনায়। Table-গুলো যুক্ত থাকে **Foreign Key** দিয়ে।

একটি সাধারণ ভুল: সব তথ্য একটি বড় Table-এ। প্রতিটি লেনদেনের সারিতে গ্রাহকের নাম ও ফোন বারবার:

normalize.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE flat (entry INTEGER, name TEXT, phone TEXT, points INTEGER);
  INSERT INTO flat VALUES (1, 'Rafi', '01711111111', 500),
    (2, 'Rafi', '01711111111', 300), (3, 'Mina', '01822222222', 400);`);

// গ্রাহক ফোন বদলালেন; একটি সারিতে হালনাগাদ হলো, অন্যটিতে নয়।
db.exec("UPDATE flat SET phone = '01799999999' WHERE entry = 1");
const phones = db.prepare(
  "SELECT DISTINCT phone FROM flat WHERE name = 'Rafi'").all();
console.log('একই Rafi-র ফোন:', phones.map((r) => r.phone).join(', '));

db.exec(`CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT, phone TEXT);
  CREATE TABLE entries (id INTEGER PRIMARY KEY,
    customer_id INTEGER REFERENCES customers (id), points INTEGER);
  INSERT INTO customers VALUES (1, 'Rafi', '01711111111'),
    (2, 'Mina', '01822222222');
  INSERT INTO entries VALUES (1, 1, 500), (2, 1, 300), (3, 2, 400);
  UPDATE customers SET phone = '01799999999' WHERE id = 1;`);
const joined = db.prepare(`SELECT c.name, c.phone, SUM(e.points) AS total
  FROM customers c JOIN entries e ON e.customer_id = c.id
  GROUP BY c.id ORDER BY c.id`).all();
console.log(joined.map((r) => `${r.name} ${r.phone} ${r.total}`).join(' | '));
```

Output

```
একই Rafi-র ফোন: 01799999999, 01711111111
Rafi 01799999999 800 | Mina 01822222222 400
```

পুনরাবৃত্ত তথ্য একসময় অমিল হবেই (**Update Anomaly**): একই গ্রাহকের দুটি ফোন নম্বর। **Normalization** মানে প্রতিটি তথ্য একবারই, নিজের Table-এ: গ্রাহকের তথ্য `customers`-এ, লেনদেন `entries`-এ, এবং দুটি `JOIN` দিয়ে মেলানো। ফোন এখন এক জায়গায় বদলায়। নিয়মগুলোর আনুষ্ঠানিক নাম আছে (First, Second, Third Normal Form), কিন্তু মূল প্রশ্ন একটিই: **এই তথ্যের মালিক কোন Table?**

উল্টো দিকও আছে: Report-এর জন্য কখনো ইচ্ছাকৃতভাবে তথ্য পুনরাবৃত্তি করা হয় (**Denormalization**), যেমন TravelData-র `accounts.balance`—Ledger-এর যোগফলের একটি Copy, যাতে প্রতিবার হাজার সারি যোগ করতে না হয়। শর্ত: Copy-টি একই Transaction-এ হালনাগাদ, এবং একটি Audit Query নিয়মিত মিলিয়ে দেখে।

২৬.৩ Constraint: Database নিজেই নিয়ম পাহারা দেয়

JavaScript-এর যাচাই এড়িয়ে যাওয়া সম্ভব: একটি পুরোনো Script, একজন Developer-এর হাতে লেখা SQL, অথবা একটি Bug! **Constraint** Database-এর ভেতরের নিয়ম, যা কোনো পথেই এড়ানো যায় না:

constraints.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`PRAGMA foreign_keys = ON;
  CREATE TABLE customers (id INTEGER PRIMARY KEY,
    email TEXT NOT NULL UNIQUE COLLATE NOCASE);
  CREATE TABLE accounts (
    customer_id INTEGER PRIMARY KEY REFERENCES customers (id),
    balance INTEGER NOT NULL
      CONSTRAINT balance_not_negative CHECK (balance >= 0));
  INSERT INTO customers VALUES (1, 'rafi@example.com');
  INSERT INTO accounts VALUES (1, 100);`);

const attempts = [
  "INSERT INTO customers VALUES (2, 'RAFI@example.com')",
  'INSERT INTO customers (id) VALUES (3)',
  'UPDATE accounts SET balance = balance - 500 WHERE customer_id = 1',
  'INSERT INTO accounts VALUES (99, 0)',
];
for (const sql of attempts) {
  try {
    db.exec(sql);
  } catch (error) {
    console.log(error.message);
  }
}
```

Output

```
UNIQUE constraint failed: customers.email
NOT NULL constraint failed: customers.email
CHECK constraint failed: balance_not_negative
FOREIGN KEY constraint failed
```

চারটি ভুল, চারটি Constraint। `COLLATE NOCASE` থাকায় বড় হাতের Email-ও একই গণ্য হয়েছে। Constraint-এর নাম (`balance_not_negative`) দিলে Error-বার্তায় নামটি আসে, যা Code ব্যবসার ভাষার বার্তায় রূপান্তর করতে পারে (Project-এর `translate`)। একটি SQLite-বিশেষ ফাঁদ: Foreign Key-র যাচাই Default বন্ধ; প্রতিটি সংযোগে `PRAGMA foreign_keys = ON` লাগে (PostgreSQL-এ সব সময় চালু)।

নিয়ম: **JavaScript-এ যাচাই ভালো বার্তার জন্য, Constraint নিশ্চয়তার জন্য।** দুটিই থাকুক।

২৬.৪ Transaction: সব, নয়তো কিছুই নয়

Point Transfer-এ দুটি লেখা: একজনের Balance কমানো, আরেকজনের বাড়ানো। প্রথমটির পরে Program বন্ধ হলে Point হাওয়া। **Transaction** কয়েকটি কাজকে একটি অবিভাজ্য একক বানায়, যার চারটি বৈশিষ্ট্য **ACID**:

বৈশিষ্ট্য	অর্থ
Atomicity	সব কাজ সফল, অথবা কোনোটিই নয়
Consistency	Transaction-এর পরে সব Constraint সত্য
Isolation	একসঙ্গে চলা Transaction একে অপরের আধা-কাজ দেখে না
Durability	`COMMIT`-এর পর তথ্য স্থায়ী, এমনকি বিদ্যুৎ গেলেও

transaction.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE accounts (id INTEGER PRIMARY KEY,
  balance INTEGER NOT NULL CHECK (balance >= 0));
  INSERT INTO accounts VALUES (1, 300), (2, 0);`);
const add = db.prepare('UPDATE accounts SET balance = balance + ? WHERE id = ?');
const balances = () => db.prepare('SELECT balance FROM accounts ORDER BY id')
  .all().map((r) => r.balance).join(' / ');

function transferUnsafe(from, to, points) {
  add.run(points, to);
  add.run(-points, from);
}

function transferSafe(from, to, points) {
  db.exec('BEGIN');
  try {
    add.run(points, to);
    add.run(-points, from);
    db.exec('COMMIT');
  } catch (error) {
    db.exec('ROLLBACK');
    throw error;
  }
}

for (const transfer of [transferUnsafe, transferSafe]) {
  db.exec('UPDATE accounts SET balance = CASE id WHEN 1 THEN 300 ELSE 0 END');
  try {
    transfer(1, 2, 500);
  } catch (error) {
```

```
    console.log(`${transfer.name}: ${error.message} | ${balances()}`);
  }
}
```

Output

```
transferUnsafe: CHECK constraint failed: balance >= 0 | 300 / 500
transferSafe: CHECK constraint failed: balance >= 0 | 300 / 0
```

Transaction ছাড়া দ্বিতীয় লেখা ব্যর্থ হলেও প্রথমটি থেকে গেছে: ৫০০ Point শূন্য থেকে তৈরি হয়েছে। Transaction-এ `ROLLBACK` দুটিকেই মুছে দিয়েছে। Project-এর `withTransaction` এই নকশাটি একবার লিখে সব জায়গায় ব্যবহার করে।

Savepoint একটি Transaction-এর ভেতরে আংশিক ফেরা: অনেকগুলো আইটেমের একটি ব্যর্থ হলে শুধু সেটি বাতিল, বাকিগুলো রেখে।

savepoint.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE gifts (name TEXT NOT NULL CHECK (length(name) >= 3));`);

const imported = [];
db.exec('BEGIN');
for (const name of ['Tea Garden Pass', 'X', 'Boat Ride']) {
  db.exec('SAVEPOINT item');
  try {
    db.prepare('INSERT INTO gifts VALUES (?)').run(name);
    db.exec('RELEASE item');
    imported.push(name);
  } catch {
    db.exec('ROLLBACK TO item');
    db.exec('RELEASE item');
  }
}
db.exec('COMMIT');
const saved = db.prepare('SELECT name FROM gifts').all().map((r) => r.name);
console.log(imported.length, saved.join(', '));
```

Output

```
2 Tea Garden Pass, Boat Ride
```

২৬.৫ Isolation, Lock এবং Lost Update

একাধিক সংযোগ একসঙ্গে একই তথ্যে কাজ করলে প্রশ্ন ওঠে: একটি Transaction অন্যটির পরিবর্তন কখন দেখবে? SQLite-এর WAL Mode-এ একটি পড়ার Transaction শুরু হওয়ার মুহূর্তের **Snapshot** দেখে, যতক্ষণ না শেষ হয়:

isolation.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';
import { mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';

const dir = mkdtempSync(join(tmpdir(), 'isolation-'));
const file = join(dir, 'demo.db');
const writer = new DatabaseSync(file);
writer.exec(`PRAGMA journal_mode = WAL;
  CREATE TABLE accounts (id INTEGER PRIMARY KEY, balance INTEGER);
  INSERT INTO accounts VALUES (1, 100);`);
const reader = new DatabaseSync(file);
const read = (db) =>
  db.prepare('SELECT balance FROM accounts WHERE id = 1').get().balance;

reader.exec('BEGIN');
console.log('Reader শুরুতে:', read(reader));
writer.exec('UPDATE accounts SET balance = 900 WHERE id = 1');
console.log('Writer-এর COMMIT-এর পরে, Reader এখনো:', read(reader));
reader.exec('COMMIT');
console.log('Reader-এর নতুন Transaction:', read(reader));

reader.close();
writer.close();
rmSync(dir, { recursive: true, force: true });
```

Output

```
Reader শুরুতে: 100
Writer-এর COMMIT-এর পরে, Reader এখনো: 100
Reader-এর নতুন Transaction: 900
```

Reader-এর Report একটি সামঞ্জস্যপূর্ণ মুহূর্তের ছবি দেখায়, মাঝপথের পরিবর্তনে বিভ্রান্ত হয় না। SQLite একসঙ্গে একজনকেই লিখতে দেয়; দ্বিতীয় Writer অপেক্ষা করে (`busy_timeout`) বা "database is locked" পায়। PostgreSQL অনেক Writer একসঙ্গে চালায় এবং চারটি **Isolation Level** দেয় (Read Uncommitted, Read Committed—Default, Repeatable Read, Serializable): কঠোর Level বেশি নিশ্চয়তা দেয়, কিন্তু বেশি সংঘাত ও Retry।

Lost Update ঘটে যখন দুটি সংযোগ একই মান পড়ে, নিজের Code-এ হিসাব করে, তারপর লেখে: দ্বিতীয় লেখা প্রথমটিকে মুছে দেয়। Project-এর `concurrency.test.mjs` দুটি আসল সংযোগে এটি ঘটিয়ে দেখায় (১০০ ও ৫০ যোগের পর ফল ৫০), তারপর তিনটি প্রতিরোধ:

১. **হিসাব Database-এর ভেতরে:** `UPDATE accounts SET balance = balance + ?` —পড়া ও লেখা একটি পরমাণু কাজ।
২. **পড়া ও লেখা একই Transaction-এ, লেখার Lock-সহ:** SQLite-এ `BEGIN IMMEDIATE`, PostgreSQL-এ `SELECT ... FOR UPDATE`।
৩. **Optimistic Concurrency:** Version মিলিয়ে হালনাগাদ (অধ্যায় ২৫)।

Deadlock: দুটি Transaction একে অপরের Lock-এর জন্য অপেক্ষা করলে কেউই এগোয় না; Database একটিকে বাতিল করে। প্রতিরোধ: সব জায়গায় একই ক্রমে Lock নেওয়া (যেমন সব সময় ছোট ID আগে), Transaction ছোট রাখা, এবং বাতিল হলে পুরো Transaction Retry।

২৬.৬ Index, Query Plan এবং N+1

Index ছাড়া "এই গ্রাহকের সব লেনদেন" খুঁজতে Database প্রতিটি সারি পড়ে (**Full Table Scan**)—দশ লাখ সারিতে দশ লাখবার। **Index** একটি সাজানো কাঠামো (সাধারণত B-tree), যা সরাসরি সঠিক জায়গায় নিয়ে যায়—অধ্যায় ১৯-এর Binary Search-এর Database রূপ। Database একটি Query কীভাবে চালাবে, তা দেখায় `EXPLAIN QUERY PLAN`; Project-এর `queries.test.mjs` থেকে আসল ফল:

queries.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
const LEDGER_QUERY = `SELECT points FROM ledger_entries
  WHERE customer_id = ? ORDER BY created_at`;

test('Index থাকলে SEARCH, Index মুছলে পুরো Table SCAN ও আলাদা সাজানো', () => {
  const { db } = memoryStore();
  const withIndex = planOf(db, LEDGER_QUERY, 1).join(' | ');
  assert.match(withIndex,
    /SEARCH ledger_entries USING INDEX ledger_customer_time/);
  db.exec('DROP INDEX ledger_customer_time');
  const without = planOf(db, LEDGER_QUERY, 1).join(' | ');
  assert.match(without, /SCAN ledger_entries/);
  assert.match(without, /USE TEMP B-TREE FOR ORDER BY/);
});


// ...
```

Index-সহ: `SEARCH ledger_entries USING INDEX ledger_customer_time (customer_id=?)` — সরাসরি খোঁজ, এবং Index-এ সারিগুলো আগে থেকেই `created_at` অনুযায়ী সাজানো, তাই আলাদা সাজাতে হয় না। Index মুছে দিলে: `SCAN ledger_entries` (পুরো Table) এবং `USE TEMP B-TREE FOR ORDER BY` (আলাদা সাজানো)। Composite Index `(customer_id, created_at)`-এর Column-এর ক্রম গুরুত্বপূর্ণ: আগে যেটি দিয়ে খোঁজা হয়, তারপর যেটি দিয়ে সাজানো।

Index-এর মূল্যও আছে: প্রতিটি `INSERT` ও `UPDATE`-এ Index-ও হালনাগাদ হয়, এবং জায়গা লাগে। তাই প্রতিটি Column-এ নয়—যে Query সত্যিই ঘনঘন চলে এবং Plan-এ যার SCAN দেখা যায়, সেখানে।

N+1 সমস্যা Index দিয়ে সমাধান হয় না—এটি Query-র সংখ্যার সমস্যা, এবং ORM ব্যবহারে প্রায়ই অজান্তে ঘটে:

n-plus-one.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE customers (id INTEGER PRIMARY KEY, name TEXT);
  CREATE TABLE entries (customer_id INTEGER, points INTEGER);`);
for (let id = 1; id <= 50; id += 1) {
  db.prepare('INSERT INTO customers VALUES (?, ?)').run(id, `Guest ${id}`);
  db.prepare('INSERT INTO entries VALUES (?, ?), (?, ?)').run(id, 100, id, 50);
}

let queries = 0;
const run = (sql, ...params) => {
  queries += 1;
  return db.prepare(sql).all(...params);
};

const slow = run('SELECT id, name FROM customers').map((c) => ({
  ...c,
  total: run('SELECT SUM(points) AS t FROM entries WHERE customer_id = ?',
    c.id)[0].t,
}));
const nPlusOne = queries;

queries = 0;
const fast = run(`SELECT c.id, c.name, SUM(e.points) AS total
  FROM customers c JOIN entries e ON e.customer_id = c.id GROUP BY c.id`);
console.log(`N+1: ${nPlusOne}টি Query, JOIN: ${queries}টি Query`);
const totals = (rows) => JSON.stringify(rows.map((r) => r.total));
console.log(totals(slow) === totals(fast));
```

Output

```
N+1: 51টি Query, JOIN: 1টি Query
true
```

একই ফল, কিন্তু ৫১ বনাম ১টি Query। একই Process-এর SQLite-এ পার্থক্য ছোট মনে হয়; Network-এর ওপারের PostgreSQL-এ প্রতিটি Query-র যাওয়া-আসায় এক মিলিসেকেন্ড বা বেশি, তাই ৫০ জন গ্রাহকের পাতায় ৫০ গুণ দেরি। ORM-এ এর সমাধান সাধারণত "Include" বা "Eager Loading", যা ভেতরে JOIN বা একটি `IN (...)` Query চালায়।

২৬.৭ Pagination, Connection Pool ও Cache

- **Keyset Pagination:** `LIMIT 20 OFFSET 100000` Database-কে এক লাখ সারি পড়ে ফেলে দিতে বাধ্য করে। `WHERE id > ? ORDER BY id LIMIT 20` Index দিয়ে সরাসরি সেখানে যায় (অধ্যায় ২৫-এর Cursor; Project-এর `byCity`)।
- **Connection Pool:** PostgreSQL-এ প্রতিটি নতুন সংযোগ ব্যয়বহুল (Authentication, Memory), এবং Server-এর সংযোগের সংখ্যা সীমিত। একটি Pool কয়েকটি সংযোগ খোলা রাখে এবং অনুরোধগুলোর মধ্যে ভাগ করে (`pg.Pool`-এর `max`)। Transaction-এর সব Query একই সংযোগে চলতে হয়—তাই `pool.connect()` দিয়ে একটি সংযোগ নিয়ে, শেষে `release()`।
- **Cache:** বারবার পড়া, কম বদলানো তথ্য (Package-এর তালিকা) Memory বা Redis-এ। কিন্তু Balance-এর মতো তথ্য Cache থেকে দেখালে বাসি হতে পারে (অধ্যায় ১৯-এর সতর্কতা); টাকা ও Point-এর সিদ্ধান্ত সব সময় Database-এর বর্তমান মান থেকে।

২৬.৮ Migration ও Backup

Schema সময়ের সঙ্গে বদলায়: নতুন Column, নতুন Index, নতুন Table। হাতে Production Database-এ `ALTER TABLE` চালানো বিপজ্জনক—কোন Database-এ কোন পরিবর্তন হয়েছে, কেউ জানে না। **Migration** প্রতিটি পরিবর্তনকে একটি ক্রমিক File-এ লেখে (`001-init.sql`, `002-indexes.sql`), Git-এ রাখে, এবং একটি Table (`schema_migrations`) মনে রাখে কোনগুলো চলেছে। TravelData-র Runner আরও দুটি নিয়ম মানে: প্রতিটি Migration নিজের Transaction-এ (মাঝপথে ব্যর্থ হলে আধা-পরিবর্তন থাকে না), এবং চলে যাওয়া File পরে বদলালে Checksum দেখে থেমে যায়—পুরোনো Migration কখনো বদলাবেন না, নতুন একটি লিখুন।

Production-এ নিরাপদ Migration-এর অভ্যাস: নতুন Column প্রথমে Default-সহ বা ঐচ্ছিক (পুরোনো Code ভাঙে না), তারপর Code হালনাগাদ, তারপর পুরোনো Column সরানো (**Expand–Contract**)। বড় Table-এ Index তৈরি Table Lock করতে পারে (PostgreSQL-এ `CREATE INDEX CONCURRENTLY`)। আর প্রতিটি Migration-এর আগে Backup।

Backup যাচাই না হলে Backup নয়। TravelData-র `backupTo` চালু Database-এর একটি সামঞ্জস্যপূর্ণ Copy নেয় (SQLite-এর `VACUUM INTO`), তারপর Copy খুলে `integrity_check` চালায় এবং প্রতিটি Table-এর সারির সংখ্যা মেলায়। PostgreSQL-এ `pg_dump`, এবং বড় System-এ Point-in-time Recovery। নিয়মিত একটি Backup থেকে আলাদা জায়গায় **Restore করে দেখুন**—না দেখা পর্যন্ত আপনি জানেন না Backup কাজ করে কি না।

২৬.৯ SQLite থেকে PostgreSQL, এবং ORM

বিষয়	SQLite (এই অধ্যায়ের চালানো Code)	PostgreSQL
চলে	একটি File, Application-এর Process-এর ভেতরে	আলাদা Server, Network-এ
একসঙ্গে লেখা	একজন Writer	অনেক Writer, Row-level Lock
Parameter	`?`	`$1`, `$2`
Auto ID	`INTEGER PRIMARY KEY AUTOINCREMENT`	`GENERATED ALWAYS AS IDENTITY`
Foreign Key	`PRAGMA foreign_keys = ON` লাগে	সব সময় চালু
Constraint Error	বার্তার লেখা	নির্দিষ্ট Code (`23505` Unique, `23514` Check, `23503` FK)
Driver	`node:sqlite` (অন্তর্নির্মিত, Synchronous)	`pg` (Package, `async`, Pool)
উপযুক্ত	Embedded, Desktop, ছোট Server, Test	বেশিরভাগ Web Backend

Project-এর `src/postgres-ledger.mjs` একই Ledger-এর `pg` সংস্করণ: `$1` Parameter, Pool থেকে একটি সংযোগে `BEGIN` / `COMMIT` / `ROLLBACK`, এবং Unique Violation (`23505`)-কে Idempotency-র "আগেই হয়েছে" হিসেবে পড়া। এটি `node --check` দিয়ে Syntax যাচাই করা হয়েছে, কিন্তু **চালানো হয়নি** (PostgreSQL Server ছিল না)।

ORM (Object-Relational Mapper), যেমন Prisma, Table-কে Object-এর মতো ব্যবহার করতে দেয়। Prisma-তে একটি Schema File-এ Model লেখা হয় (`model Customer { id Int @id @default(autoincrement()) email String @unique ... }`), তা থেকে Migration ও একটি Type-যুক্ত Client তৈরি হয় (অধ্যায় ২৪-এর TypeScript-এর সুবিধা):

prisma-example.mjs · JavaScript চলবে: Node.js

```
// উদাহরণ মাত্র: @prisma/client ও একটি Database প্রয়োজন। এই বইয়ের QA-তে
// Prisma Install বা চালানো হয়নি।
import { PrismaClient } from '@prisma/client';

const prisma = new PrismaClient();

const rich = await prisma.customer.findMany({
  where: { account: { balance: { gte: 5_000 } } },
  include: { account: true },
  orderBy: { id: 'asc' },
  take: 20,
});
```

‘‘

```
await prisma.$transaction(async (tx) => {
  await tx.account.update({
    where: { customerId: 1 }, data: { balance: { decrement: 500 } },
  });
  await tx.account.update({
    where: { customerId: 2 }, data: { balance: { increment: 500 } },
  });
});
console.log(rich.length);
```

ORM-এর সুবিধা	ORM-এর ঝুঁকি
Type-যুক্ত Query, কম Boilerplate	Generate হওয়া SQL অদৃশ্য; N+1 সহজে ঘটে
Migration-এর Tool	জটিল Report-এ অদক্ষ Query
Parameter স্বয়ংক্রিয় (Injection-নিরাপদ)	`$queryRawUnsafe`-এর মতো ফাঁক
Database বদলানো তুলনামূলক সহজ	Database-এর বিশেষ সুবিধা (Lock, CTE) কঠিন

পরামর্শ: ORM ব্যবহার করলেও SQL শিখুন, এবং ধীর Query-র আসল SQL ও Plan দেখুন। অনেক দল দুটি একসঙ্গে ব্যবহার করে: সাধারণ CRUD-এ ORM, জটিল Report-এ হাতে লেখা Parameterized SQL।

২৬.১০ Security ও Observability

- **Least Privilege:** Application-এর Database ব্যবহারকারী শুধু প্রয়োজনীয় Table-এ `SELECT/INSERT/UPDATE`; Migration-এর জন্য আলাদা, বেশি অধিকারের ব্যবহারকারী।
- **Secret:** `DATABASE_URL` Environment বা Secret Manager-এ, কখনো Git-এ নয় (অধ্যায় ২০)।
- **Parameterized Query সব সময়,** ORM-এর Raw Query-তেও।
- **Encryption:** Network-এ TLS; সংবেদনশীল Column (যেমন Passport নম্বর) আলাদাভাবে Encrypt করার কথা ভাবুন।
- **Observability:** ধীর Query-র Log (PostgreSQL-এর `log_min_duration_statement`), Pool-এর ব্যবহার, Lock-এর অপেক্ষা, এবং নিয়মিত Audit Query (TravelData-র `audit()`)।

২৬.১১ Project: TravelData Pro

TravelData Pro একটি ভ্রমণ সংস্থার গ্রাহক ও Loyalty Point-এর Data Layer: গ্রাহক নিবন্ধন (Account-সহ, একটি Transaction-এ), Point অর্জন, ব্যয় ও পরিবারে হস্তান্তর (Append-only Ledger, Idempotency Key), Report, Query Plan, Migration ও যাচাই-করা Backup। v30-এর TravelData Pro-র নাম রাখা হয়েছে; Domain হিসেবে

Booking-এর বদলে Loyalty Ledger বেছে নেওয়া হয়েছে, যাতে আগের অধ্যায়গুলোর Booking Project-এর পুনরাবৃত্তি না হয় এবং Transaction-এর নিয়মগুলো স্পষ্ট দেখা যায়।

> **নোট:** শিক্ষামূলক Project। SQLite-এ চালানো ও পরীক্ষিত; PostgreSQL Adapter শুধু Syntax-যাচাই করা। কোনো আসল গ্রাহক বা কোম্পানির Database-এ যুক্ত নয়; সব Test নিজস্ব Memory বা Temp Database তৈরি করে।

File-এর কাঠামো

Folder-এর কাঠামো

```
travel-data/
├── demo.mjs
├── migrations/             001-init.sql, 002-indexes.sql, 003-tier.sql
├── src/
│   ├── db.mjs              সংযোগের Setting, withTransaction, Constraint চেনা
│   ├── migrate.mjs         Migration Runner (Checksum)
│   ├── errors.mjs          DataError, translate
│   ├── customers.mjs       গ্রাহক, Keyset Pagination, নিরাপদ খোঁজ
│   ├── ledger.mjs          earn, redeem, transfer, audit
│   ├── reports.mjs         Report ও Query Plan
│   ├── backup.mjs          VACUUM INTO ও যাচাই
│   └── postgres-ledger.mjs PostgreSQL সংস্করণ (চালানো হয়নি)
└── tests/                  schema, ledger, concurrency, queries (+ helpers)
```

Migration

001-init.sql · SQL — চলবে: Node.js

```sql
-- গ্রাহক, তাঁর Loyalty Account, এবং প্রতিটি Point-এর লেনদেন (Ledger)।
CREATE TABLE customers (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL
    CONSTRAINT name_length CHECK (length(trim(name)) BETWEEN 2 AND 80),
  email TEXT NOT NULL UNIQUE COLLATE NOCASE,
  phone TEXT NOT NULL UNIQUE
    CONSTRAINT phone_format CHECK (length(phone) = 11
      AND phone GLOB '01[3-9]*' AND phone NOT GLOB '*[^0-9]*'),
  city TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%fZ', 'now'))
);

CREATE TABLE accounts (
  customer_id INTEGER PRIMARY KEY REFERENCES customers (id) ON DELETE CASCADE,
  balance INTEGER NOT NULL DEFAULT 0
    CONSTRAINT balance_not_negative CHECK (balance >= 0)
);
```

```sql
CREATE TABLE ledger_entries (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  customer_id INTEGER NOT NULL REFERENCES customers (id),
  points INTEGER NOT NULL CONSTRAINT points_not_zero CHECK (points <> 0),
  reason TEXT NOT NULL,
  idempotency_key TEXT UNIQUE,
  created_at TEXT NOT NULL
);
```

002-indexes.sql · SQL চলবে: Node.js

```sql
-- প্রায়ই চলা Query-র জন্য Index: একজন গ্রাহকের লেনদেন সময়ের ক্রমে,
-- এবং শহর ধরে গ্রাহক।
CREATE INDEX ledger_customer_time ON ledger_entries (customer_id, created_at);
CREATE INDEX customers_city ON customers (city);
```

003-tier.sql · SQL চলবে: Node.js

```sql
-- নতুন Column, Default-সহ (পুরোনো সারিও বৈধ থাকে), এবং বিদ্যমান তথ্য থেকে
-- মান পূরণ (Data Migration)।
ALTER TABLE accounts ADD COLUMN tier TEXT NOT NULL DEFAULT 'silver'
  CONSTRAINT tier_known CHECK (tier IN ('silver', 'gold', 'platinum'));

UPDATE accounts SET tier = CASE
  WHEN balance >= 20000 THEN 'platinum'
  WHEN balance >= 5000 THEN 'gold'
  ELSE 'silver'
END;
```

তিনটি Table-এর প্রতিটি নিয়ম Constraint-এ: নামের দৈর্ঘ্য, ফোনের রূপ, Email ও ফোন অনন্য, Balance কখনো ঋণাত্মক নয়, Point শূন্য নয়, এবং প্রতিটি লেনদেনের Idempotency Key অনন্য। `003-tier.sql` দেখায় Production-বান্ধব পরিবর্তন: নতুন Column একটি Default-সহ (পুরোনো সারি বৈধ থাকে), তারপর বিদ্যমান তথ্য থেকে মান পূরণ (Data Migration)। লক্ষ করুন, `tier` এখানে একবারই হিসাব হয়; Balance বদলালে স্বয়ংক্রিয়ভাবে বদলায় না—বাস্তবে এটি একটি নিয়মিত কাজ বা Trigger-এর দায়িত্ব।

src/db.mjs, src/migrate.mjs ও src/errors.mjs

db.mjs · JavaScript চলবে: Node.js

```javascript
import { DatabaseSync } from 'node:sqlite';

// প্রতিটি সংযোগে একই Setting: Foreign Key চালু (SQLite-এ Default বন্ধ!),
// WAL Journal (পড়া ও লেখা একসঙ্গে), এবং ব্যস্ত থাকলে কতক্ষণ অপেক্ষা।
export function openDatabase(file = ':memory:', options = {}) {
  const { busyTimeoutMs = 2_000 } = options;
  const db = new DatabaseSync(file);
  db.exec('PRAGMA foreign_keys = ON');
```

```
  if (file !== ':memory:') {
    db.exec('PRAGMA journal_mode = WAL');
  }
  db.exec(`PRAGMA busy_timeout = ${Number(busyTimeoutMs)}`);
  return db;
}

// fn-এর সব কাজ একসঙ্গে সফল, নয়তো কিছুই নয় (Atomicity)।
// IMMEDIATE: শুরুতেই লেখার Lock, যাতে মাঝপথে অন্য Writer-এর সঙ্গে সংঘাত না হয়।
export function withTransaction(db, fn) {
  db.exec('BEGIN IMMEDIATE');
  try {
    const result = fn();
    db.exec('COMMIT');
    return result;
  } catch (error) {
    db.exec('ROLLBACK');
    throw error;
  }
}

// SQLite-এর Constraint Error-কে অর্থপূর্ণ Code-এ রূপান্তর।
export function constraintOf(error) {
  const message = String(error?.message ?? '');
  if (message.includes('UNIQUE constraint failed')) {
    return { code: 'UNIQUE', column: message.split(': ').at(-1) };
  }
  if (message.includes('CHECK constraint failed')) {
    return { code: 'CHECK', column: message.split(': ').at(-1) };
  }
  if (message.includes('FOREIGN KEY constraint failed')) {
    return { code: 'FOREIGN_KEY', column: null };
  }
  return null;
}
```

migrate.mjs · JavaScript চলবে: Node.js

```
import { readdirSync, readFileSync } from 'node:fs';
import { createHash } from 'node:crypto';
import { join } from 'node:path';
import { fileURLToPath } from 'node:url';
import { withTransaction } from './db.mjs';

const DEFAULT_DIR = fileURLToPath(new URL('../migrations/', import.meta.url));

// সংস্করণভিত্তিক Migration: প্রতিটি File একবারই, ক্রমে, নিজস্ব Transaction-এ।
// চলে যাওয়া File পরে বদলালে (Checksum আলাদা) থেমে যায়।
export function migrate(db, dir = DEFAULT_DIR) {
```

```
  db.exec(`CREATE TABLE IF NOT EXISTS schema_migrations (
    version TEXT PRIMARY KEY, checksum TEXT NOT NULL,
    applied_at TEXT NOT NULL)`);
  const rows = db.prepare(
    'SELECT version, checksum FROM schema_migrations').all();
  const applied = new Map(rows.map((row) => [row.version, row.checksum]));
  const files = readdirSync(dir)
    .filter((f) => /^\d{3}-.+\.sql$/.test(f)).sort();
  const ran = [];
  for (const file of files) {
    const sql = readFileSync(join(dir, file), 'utf8');
    const checksum = createHash('sha256').update(sql).digest('hex');
    const version = file.slice(0, 3);
    if (applied.has(version)) {
      if (applied.get(version) !== checksum) {
        throw new Error(`Migration ${file} চলার পরে বদলানো হয়েছে`);
      }
      continue;
    }
    withTransaction(db, () => {
      db.exec(sql);
      db.prepare('INSERT INTO schema_migrations VALUES (?, ?, ?)')
        .run(version, checksum, new Date().toISOString());
    });
    ran.push(file);
  }
  return ran;
}
```

errors.mjs · JavaScript চলবে: Node.js

```
import { constraintOf } from './db.mjs';

export class DataError extends Error {
  constructor(code, message) {
    super(message);
    this.name = 'DataError';
    this.code = code;
  }
}

// Database-এর Constraint Error থেকে ব্যবসার ভাষার Error। অজানা হলে যেমন ছিল।
export function translate(error, messages = {}) {
  const constraint = constraintOf(error);
  if (!constraint) {
    return error;
  }
  const key = constraint.column ?? constraint.code;
  const message = messages[key] ?? messages[constraint.code] ?? error.message;
```

```
  return new DataError(constraint.code, message);
}
```

`withTransaction` `BEGIN IMMEDIATE` ব্যবহার করে: Transaction শুরুতেই লেখার Lock নেয়, তাই পড়ার পরে লেখার মুহূর্তে অন্য Writer-এর সঙ্গে সংঘাত হয় না (একটি সাধারণ `BEGIN` পড়ার পরে লেখায় "উন্নীত" হতে গিয়ে ব্যর্থ হতে পারে)। `constraintOf` SQLite-এর বার্তা থেকে Constraint-এর ধরন ও নাম বের করে; `translate` সেগুলো ব্যবসার ভাষায় বদলায়।

src/customers.mjs

customers.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { withTransaction } from './db.mjs';
import { translate } from './errors.mjs';

const MESSAGES = {
  'customers.email': 'এই Email-এ আগেই একজন গ্রাহক আছেন',
  'customers.phone': 'এই ফোন নম্বরে আগেই একজন গ্রাহক আছেন',
  name_length: 'নাম ২ থেকে ৮০ অক্ষর',
  phone_format: '১১ অঙ্কের মোবাইল নম্বর',
};

// সব Query Parameterized; LIKE-এর বিশেষ চিহ্ন (% ও _) Escape করা।
const escapeLike = (text) => text.replace(/[\\%_]/g, (ch) => `\\${ch}`);

export function createCustomerStore(db) {
  const sql = {
    insert: db.prepare(
      'INSERT INTO customers (name, email, phone, city) VALUES (?, ?, ?, ?)'),
    account: db.prepare('INSERT INTO accounts (customer_id) VALUES (?)'),
    byEmail: db.prepare('SELECT * FROM customers WHERE email = ?'),
    page: db.prepare(`SELECT id, name FROM customers
      WHERE city = ? AND id > ? ORDER BY id LIMIT ?`),
    search: db.prepare(`SELECT id, name FROM customers
      WHERE name LIKE ? ESCAPE '\\' ORDER BY name, id LIMIT 20`),
  };

// ...
```

customers.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

  // গ্রাহক ও তাঁর Account একসঙ্গে: একটি ব্যর্থ হলে দুটিই বাতিল।
  function create({ name, email, phone, city }) {
    try {
      return withTransaction(db, () => {
        const { lastInsertRowid } = sql.insert.run(name, email, phone, city);
```

```
        sql.account.run(lastInsertRowid);
        return Number(lastInsertRowid);
      });
    } catch (error) {
      throw translate(error, MESSAGES);
    }
  }

  // Keyset Pagination: "এই id-র পর থেকে", Index দিয়ে সরাসরি।
  const byCity = (city, { after = 0, limit = 20 } = {}) =>
    sql.page.all(city, after, limit).map((row) => ({ ...row }));

  const search = (text) =>
    sql.search.all(`%${escapeLike(text)}%`).map((row) => ({ ...row }));

  const findByEmail = (email) => {
    const row = sql.byEmail.get(email);
    return row ? { ...row } : null;
  };

  return { create, byCity, search, findByEmail };
}
```

src/ledger.mjs

ledger.mjs · JavaScript অংশ ১ / ২ (Part 1 of 2) চলবে: Node.js

```
import { withTransaction } from './db.mjs';
import { DataError, translate } from './errors.mjs';

const MESSAGES = {
  balance_not_negative: 'যথেষ্ট Point নেই',
  FOREIGN_KEY: 'গ্রাহক নেই',
};

// Ledger: প্রতিটি পরিবর্তন একটি নতুন সারি, কখনো মুছে বা বদলে নয় (Append-only)।
// balance তার যোগফলের একটি দ্রুত Copy, একই Transaction-এ হালনাগাদ।
export function createLedger(db, { now = () => new Date() } = {}) {
  const sql = {
    entry: db.prepare(`INSERT INTO ledger_entries
      (customer_id, points, reason, idempotency_key, created_at)
      VALUES (?, ?, ?, ?, ?)`),
    adjust: db.prepare(
      'UPDATE accounts SET balance = balance + ? WHERE customer_id = ?'),
    seen: db.prepare('SELECT 1 FROM ledger_entries WHERE idempotency_key = ?'),
    balance: db.prepare(
      'SELECT balance, tier FROM accounts WHERE customer_id = ?'),
    audit: db.prepare(`SELECT a.customer_id AS id, a.balance,
      COALESCE(SUM(e.points), 0) AS total
```

```
      FROM accounts a
      LEFT JOIN ledger_entries e ON e.customer_id = a.customer_id
      GROUP BY a.customer_id HAVING a.balance <> total`),
  };

  function post(customerId, points, reason, key) {
    sql.entry.run(customerId, points, reason, key, now().toISOString());
    if (sql.adjust.run(points, customerId).changes !== 1) {
      throw new DataError('NOT_FOUND', 'গ্রাহকের Account নেই');
    }
  }

// ...
```

ledger.mjs · JavaScript অংশ ২ / ২ (Part 2 of 2) চলবে: Node.js

```
// ...

  // একই key আগে দেখা গেলে কিছুই না করে duplicate: true (নিরাপদ Retry)।
  function run(key, steps) {
    if (!Number.isInteger(steps[0].points) || steps[0].points === 0) {
      throw new DataError('INVALID', 'Point শূন্য নয় এমন পূর্ণসংখ্যা');
    }
    try {
      return withTransaction(db, () => {
        if (key && sql.seen.get(`${key}:0`)) {
          return { duplicate: true };
        }
        steps.forEach((s, i) => post(s.customerId, s.points, s.reason,
          key ? `${key}:${i}` : null));
        return { duplicate: false };
      });
    } catch (error) {
      throw translate(error, MESSAGES);
    }
  }

  const earn = (customerId, points, reason, key) =>
    run(key, [{ customerId, points, reason }]);
  const redeem = (customerId, points, reason, key) =>
    run(key, [{ customerId, points: -points, reason }]);
  // এক গ্রাহক থেকে আরেকজনে: দুটি সারি, একটি Transaction—আধা-সম্পূর্ণ হয় না।
  const transfer = (from, to, points, key) => run(key, [
    { customerId: from, points: -points, reason: `transfer to ${to}` },
    { customerId: to, points, reason: `transfer from ${from}` },
  ]);

  const balanceOf = (customerId) => {
    const row = sql.balance.get(customerId);
```

```
    return row ? { ...row } : null;
  };
  // Invariant: প্রতিটি balance তার Ledger-এর যোগফলের সমান। মিল না থাকলে তালিকা।
  const audit = () => sql.audit.all().map((row) => ({ ...row }));

  return { earn, redeem, transfer, balanceOf, audit };
}
```

Ledger-এর তিনটি নকশার সিদ্ধান্ত। **Append-only:** কোনো লেনদেন বদলানো বা মোছা হয় না; ভুল হলে উল্টো একটি লেনদেন—হিসাবরক্ষণের প্রাচীন নিয়ম, যাতে পুরো ইতিহাস থাকে। **Idempotency Database-এ:** Key আগে দেখা হয়, এবং দুটি অনুরোধ একসঙ্গে এলেও Ledger-এর `UNIQUE` Constraint দ্বিতীয়টিকে আটকায়—Memory-র `Map` (অধ্যায় ২৫) একাধিক Server-এ যা পারে না। **Invariant ও Audit:** প্রতিটি Balance তার Ledger-এর যোগফলের সমান; `audit()` একটি Query-তে অমিল খোঁজে, এবং ৫০০টি এলোমেলো কাজের পরেও (Property Test) তালিকা খালি।

src/reports.mjs, src/backup.mjs ও src/postgres-ledger.mjs

reports.mjs · JavaScript চলবে: Node.js

```
// Report-এর Query: JOIN, GROUP BY, এবং Query Plan দেখার সাহায্যকারী।
export function topCustomers(db, limit = 5) {
  return db.prepare(`SELECT c.name, c.city, a.balance, a.tier
    FROM customers c JOIN accounts a ON a.customer_id = c.id
    ORDER BY a.balance DESC, c.id LIMIT ?`)
    .all(limit).map((row) => ({ ...row }));
}

export function monthlyPoints(db, customerId) {
  return db.prepare(`SELECT substr(created_at, 1, 7) AS month,
      SUM(CASE WHEN points > 0 THEN points ELSE 0 END) AS earned,
      SUM(CASE WHEN points < 0 THEN -points ELSE 0 END) AS spent
    FROM ledger_entries WHERE customer_id = ?
    GROUP BY month ORDER BY month`).all(customerId).map((row) => ({ ...row }));
}

// EXPLAIN QUERY PLAN: Database Query-টি কীভাবে চালাবে (Index, না পুরো Table)।
export function planOf(db, sql, ...params) {
  return db.prepare(`EXPLAIN QUERY PLAN ${sql}`).all(...params)
    .map((row) => row.detail);
}
```

backup.mjs · JavaScript চলবে: Node.js

```
import { existsSync } from 'node:fs';
import { DatabaseSync } from 'node:sqlite';

// চালু Database-এর একটি সামঞ্জস্যপূর্ণ Copy (VACUUM INTO), তারপর যাচাই:
// Backup খোলা যায়, ক্ষতিগ্রস্ত নয়, এবং প্রতিটি Table-এ সারির সংখ্যা মেলে।
```

```
export function backupTo(db, file) {
  if (existsSync(file)) {
    throw new Error(`${file} আগেই আছে; পুরোনো Backup মুছে যাবে না`);
  }
  db.prepare('VACUUM INTO ?').run(file);
  const copy = new DatabaseSync(file, { readOnly: true });
  try {
    const check = copy.prepare('PRAGMA integrity_check').get();
    const integrity = check.integrity_check;
    const counts = {};
    for (const table of ['customers', 'accounts', 'ledger_entries']) {
      const sql = `SELECT COUNT(*) AS n FROM ${table}`;
      const original = db.prepare(sql).get().n;
      const backedUp = copy.prepare(sql).get().n;
      if (original !== backedUp) {
        throw new Error(`${table}: ${original} বনাম ${backedUp} সারি`);
      }
      counts[table] = backedUp;
    }
    return { integrity, counts };
  } finally {
    copy.close();
  }
}
```

postgres-ledger.mjs · JavaScript · অংশ ১ / ২ (Part 1 of 2) — চলবে: Node.js

```
// PostgreSQL-এর জন্য একই Ledger-এর চুক্তি (pg Driver, async)।
// এই বইয়ের QA-তে চালানো হয়নি: কোনো PostgreSQL Server ছিল না; শুধু Syntax যাচাই।
import pg from 'pg';

export const createPool = (connectionString) =>
  new pg.Pool({ connectionString, max: 10, idleTimeoutMillis: 30_000 });

// ...
```

postgres-ledger.mjs · JavaScript · অংশ ২ / ২ (Part 2 of 2) — চলবে: Node.js

```
// ...

// PostgreSQL-এর Error Code: 23505 Unique, 23514 Check, 23503 Foreign Key।
export function createPgLedger(pool) {
  async function run(key, steps) {
    const client = await pool.connect();
    try {
      await client.query('BEGIN');
      for (const [i, step] of steps.entries()) {
        await client.query(`INSERT INTO ledger_entries
          (customer_id, points, reason, idempotency_key, created_at)
```

```
          VALUES ($1, $2, $3, $4, now())`,
        [step.customerId, step.points, step.reason,
          key ? `${key}:${i}` : null]);
        const { rowCount } = await client.query(
          'UPDATE accounts SET balance = balance + $1 WHERE customer_id = $2',
          [step.points, step.customerId]);
        if (rowCount !== 1) {
          throw new Error('গ্রাহকের Account নেই');
        }
      }
      await client.query('COMMIT');
      return { duplicate: false };
    } catch (error) {
      await client.query('ROLLBACK');
      if (error.code === '23505' && key) {
        return { duplicate: true };
      }
      throw error;
    } finally {
      client.release();
    }
  }

  return {
    earn: (customerId, points, reason, key) =>
      run(key, [{ customerId, points, reason }]),
    redeem: (customerId, points, reason, key) =>
      run(key, [{ customerId, points: -points, reason }]),
  };
}
```

demo.mjs

demo.mjs · JavaScript চলবে: Node.js

```
import { mkdirSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { openDatabase } from './src/db.mjs';
import { migrate } from './src/migrate.mjs';
import { createCustomerStore } from './src/customers.mjs';
import { createLedger } from './src/ledger.mjs';
import { topCustomers, monthlyPoints } from './src/reports.mjs';
import { backupTo } from './src/backup.mjs';

// শিক্ষামূলক Demo: Database File Operating System-এর Temp Folder-এ, প্রতিবার নতুন।
const dir = join(tmpdir(), 'travel-data-demo');
rmSync(dir, { recursive: true, force: true });
mkdirSync(dir);
```

```
const db = openDatabase(join(dir, 'travel.db'));

console.log('Migration:', migrate(db).join(', '));
const customers = createCustomerStore(db);
const now = () => new Date('2026-10-18T10:00:00Z');
const ledger = createLedger(db, { now });
const rafi = customers.create({ name: 'Rafi Ahmed', email: 'rafi@example.com',
  phone: '01712345678', city: 'Dhaka' });
const mina = customers.create({ name: 'Mina Akter', email: 'mina@example.com',
  phone: '01812345678', city: 'Sylhet' });

ledger.earn(rafi, 6_000, "Cox's Bazar tour", 'trip-1001');
ledger.earn(rafi, 6_000, "Cox's Bazar tour", 'trip-1001');
ledger.transfer(rafi, mina, 1_500, 'family-77');
try {
  ledger.redeem(mina, 9_000, 'Free hotel night');
} catch (error) {
  console.log(`${error.code}: ${error.message}`);
}

console.log(topCustomers(db).map((c) => `${c.name} ${c.balance}`).join(' | '));
console.log(JSON.stringify(monthlyPoints(db, rafi)));
console.log('Audit:', ledger.audit().length === 0 ? 'balance ও Ledger মেলে' :
    'অমিল!');
const { integrity, counts } = backupTo(db, join(dir, 'backup.db'));
console.log(`Backup: ${integrity}, ${JSON.stringify(counts)}`);
db.close();
```

Output

```
Migration: 001-init.sql, 002-indexes.sql, 003-tier.sql
CHECK: যথেষ্ট Point নেই
Rafi Ahmed 4500 | Mina Akter 1500
[{"month":"2026-10","earned":6000,"spent":1500}]
Audit: balance ও Ledger মেলে
Backup: ok, {"customers":2,"accounts":2,"ledger_entries":3}
```

একই Tour-এর Point (`trip-1001`) দুবার পাঠানো হলেও একবারই যোগ হয়েছে; পরিবারে ১,৫০০ Point হস্তান্তর; Mina-র ৯,০০০ Point ব্যয়ের চেষ্টা Constraint-এ আটকে Rollback। Audit মিলেছে, এবং Backup যাচাই করা।

Test: Transaction ও Concurrency

ledger.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('transfer পরমাণু: ব্যর্থ হলে দুই পক্ষই অপরিবর্তিত', () => {
  const { db, customers, ledger } = memoryStore();
```

```
  const [a, b] = [newCustomer(customers), newCustomer(customers)];
  ledger.earn(a, 1_000, 'tour');
  assert.throws(() => ledger.transfer(a, b, 5_000), { code: 'CHECK' });
  const both = () => [ledger.balanceOf(a).balance, ledger.balanceOf(b).balance];
  assert.deepEqual(both(), [1_000, 0]);
  assert.throws(() => ledger.transfer(a, 999, 100), { code: 'FOREIGN_KEY' });
  assert.equal(ledger.balanceOf(a).balance, 1_000);
  assert.equal(entries(db), 1);
  ledger.transfer(a, b, 400);
  assert.deepEqual(both(), [600, 400]);
});

// ...
```

concurrency.test.mjs · JavaScript নির্বাচিত অংশ (Selected Excerpt — Complete file in Companion Code) চলবে: Node.js

```
// ...

test('একজন লিখছেন: দ্বিতীয় Writer "database is locked" পান, পরে সফল', (t) => {
  const { first, second } = twoConnections(t);
  first.exec('BEGIN IMMEDIATE');
  assert.throws(() => second.exec('BEGIN IMMEDIATE'), /database is locked/);
  first.exec('COMMIT');
  second.exec('BEGIN IMMEDIATE');
  second.exec('COMMIT');
});

test('Lost Update: পড়ে, হিসাব করে, লেখা (দুই সংযোগে) একটি হালনাগাদ হারায়', (t) => {
  const { first, second, id } = twoConnections(t);
  const readA = balance(first, id);
  const readB = balance(second, id);
  first.prepare(SET).run(readA + 100, id);
  second.prepare(SET).run(readB + 50, id);
  assert.equal(balance(first, id), 50);
});

// ...
```

প্রথম Test দেখায় Transfer-এর পরমাণুতা: যথেষ্ট Point না থাকলে, বা প্রাপক না থাকলে, দুই পক্ষই অপরিবর্তিত এবং Ledger-এ কোনো আধা-সারি নেই। দ্বিতীয়টি একই Database File-এ দুটি আসল সংযোগ খোলে: একজন `BEGIN IMMEDIATE` ধরে রাখলে দ্বিতীয়জন "database is locked" পান; এবং পড়ে-হিসাব-করে-লেখা দুই সংযোগে চালালে ১০০ + ৫০-এর বদলে ফল ৫০—Lost Update প্রমাণিত, তারপর পরের দুটি Test-এ তার দুটি সমাধান।

চালানো এবং প্রত্যাশিত আচরণ

Terminal

```
cd travel-data
node --test
node demo.mjs
```

`node --test` ২১টি Test চালায়: `schema` ৬ (Migration-এর ক্রম ও Checksum, প্রতিটি Constraint, Foreign Key, ব্যর্থ create-এ কিছুই না থাকা, Data Migration), `ledger` ৬ (earn/redeem, Rollback, পরমাণু Transfer, Idempotency, অবৈধ Point, ৫০০ কাজের Property), `concurrency` ৪ (Lock, Lost Update ও দুটি সমাধান), এবং `queries` ৫ (Query Plan, Keyset Pagination, নিরাপদ খোঁজ, Report, Backup); শেষে `# pass 21`, `# fail 0`। `node:sqlite` একটি "ExperimentalWarning" দেখায়—স্বাভাবিক। Node.js 22.5 বা নতুন লাগে; কোনো Package লাগে না।

সমস্যা হলে যা দেখবেন

লক্ষণ	সম্ভাব্য কারণ	করণীয়
Foreign Key ভুল সত্ত্বেও লেখা সফল	`PRAGMA foreign_keys` বন্ধ	প্রতিটি সংযোগে `openDatabase`
"database is locked"	অন্য সংযোগ লেখার Transaction ধরে আছে	Transaction ছোট, `busy_timeout`, Retry
Migration থেমে যায়: "চলার পরে বদলানো"	পুরোনো Migration File সম্পাদিত	পরিবর্তনটি নতুন Migration-এ
Query ধীর	Plan-এ SCAN	উপযুক্ত Index, তারপর আবার Plan
Audit-এ অমিল	কেউ Transaction ছাড়া বা সরাসরি SQL-এ Balance বদলেছে	Ledger-এর বাইরে লেখা বন্ধ
Windows-এ Test-এর Temp File মোছা যায় না	Database খোলা	আগে `close()`, তারপর মোছা

২৬.১২ সাধারণ ভুল

ভুল	কেন সমস্যা	সমাধান
শুধু JavaScript-এ যাচাই	অন্য পথে ভুল তথ্য ঢোকে	Constraint
একাধিক লেখা Transaction ছাড়া	আধা-সম্পূর্ণ অবস্থা	`withTransaction`
পড়ে, Code-এ হিসাব, তারপর লেখা	Lost Update	`balance = balance + ?`, Lock, Version
প্রতিটি সারির জন্য আলাদা Query	N+1	JOIN, Eager Loading

ভুল	কেন সমস্যা	সমাধান
প্রতিটি Column-এ Index	লেখা ধীর, জায়গা বেশি	Plan দেখে নির্বাচিত Index
`OFFSET` দিয়ে গভীর পাতা	প্রতিবার হাজারো সারি বাদ	Keyset Pagination
চলে যাওয়া Migration সম্পাদনা	Database-ভেদে ভিন্ন Schema	নতুন Migration
যাচাই-না-করা Backup	দরকারের দিন অকেজো	Restore করে দেখা
SQLite-এ Foreign Key ধরে নেওয়া	Default বন্ধ	`PRAGMA foreign_keys = ON`
ORM-এর Generate করা SQL কখনো না দেখা	লুকানো ধীর Query	Query Log ও Plan

২৬.১৩ Debugging নির্দেশিকা

১. **Constraint Error:** বার্তার Table, Column বা Constraint-এর নাম পড়ুন; Code-এর যাচাই আর Database-এর নিয়ম মেলে কি না দেখুন।
২. **ধীর Query:** `EXPLAIN QUERY PLAN` (PostgreSQL-এ `EXPLAIN ANALYZE`); SCAN ও TEMP B-TREE খুঁজুন।
৩. **Query-র সংখ্যা গুনুন:** একটি পাতায় কতগুলো Query চলে (N+1)।
৪. **Lock:** কোন Transaction দীর্ঘ সময় খোলা; Transaction-এর ভেতরে Network Call বা ব্যবহারকারীর অপেক্ষা নেই তো?
৫. **তথ্যের অমিল:** Audit Query; Ledger থেকে আবার হিসাব।
৬. **Migration:** `schema_migrations` Table দেখুন—কোনগুলো চলেছে, কখন।

২৬.১৪ পেশাদার অভ্যাস

- **প্রতিটি ব্যবসার নিয়ম, যতটা সম্ভব, একটি নামযুক্ত Constraint-এ।**
- **একসঙ্গে বদলাতে হবে এমন সব লেখা একটি Transaction-এ;** Transaction ছোট।
- **টাকা ও Point: Append-only Ledger, Integer, এবং Audit।**
- **Idempotency Key-র জন্য UNIQUE Constraint।**
- **প্রতিটি Schema পরিবর্তন একটি Migration,** Git-এ, Code Review-সহ।
- **Index Plan দেখে, মেপে।**
- **Backup স্বয়ংক্রিয়, এবং নিয়মিত Restore-এর অনুশীলন।**
- **Test-এ আসল Database (Memory বা Temp File),** নকল নয়—Constraint ও Transaction-এর আচরণ নকল করা যায় না।

২৬.১৫ অনুশীলনী

প্রাথমিক স্তর

১. `tours` Table-এর জন্য Constraint লিখুন: `price` ধনাত্মক, `days` ১ থেকে ৩০, `code` অনন্য।

২. Normalization-এর আগে-পরের দুটি Table-এর নকশা করুন: `bookings`-এ বারবার থাকা Tour-এর নাম ও দাম।

৩. ACID-এর প্রতিটি অক্ষর এক বাক্যে।

৪. `SELECT * FROM customers WHERE email = ?`-এর জন্য কোন Index? Plan দিয়ে যাচাই করুন।

৫. N+1 সমস্যা কী? এক বাক্যে, একটি উদাহরণসহ।

মধ্যম স্তর

৬. Ledger-এ `reverse(entryId)` যোগ করুন: একটি লেনদেনের উল্টো লেনদেন, মূলটি না বদলে।

৭. একটি Migration `004-referral.sql` লিখুন: `customers`-এ ঐচ্ছিক `referred_by` (অন্য গ্রাহকের Foreign Key)।

৮. "প্রতিটি শহরের মোট Point" Report-এর Query লিখুন।

৯. একটি Transaction-এর ভেতরে তিনটি গ্রাহক Import করুন, যার একটি অবৈধ; Savepoint দিয়ে শুধু সেটি বাদ দিন।

চ্যালেঞ্জ স্তর

১০. Lock না পেলে ৩ বার Retry করা `withRetry(fn)` লিখুন, এবং দুটি সংযোগে পরীক্ষা করুন।

১১. Ledger থেকে সব Balance নতুন করে হিসাব করার একটি "Repair" Query লিখুন, এবং কেন এটি শুধু জরুরি অবস্থায়—ব্যাখ্যা করুন।

১২. LeadDesk (অধ্যায় ২৫)-এর Repository-কে Migration ও Constraint-সহ SQLite-এ নেওয়ার নকশা করুন: Table, Index, Idempotency ও Version।

২৬.১৬ অনুশীলনীর সমাধান

সমাধান ১

tour-constraints.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec(`CREATE TABLE tours (
  code TEXT PRIMARY KEY,
```

```
  price INTEGER NOT NULL CONSTRAINT price_positive CHECK (price > 0),
  days INTEGER NOT NULL CONSTRAINT days_range CHECK (days BETWEEN 1 AND 30))`);

const tries = [
  ['SJK-2', 6500, 2], ['SJK-2', 7000, 3], ['KTA-0', 0, 2], ['LNG-31', 9000, 31],
];
for (const row of tries) {
  try {
    db.prepare('INSERT INTO tours VALUES (?, ?, ?)').run(...row);
    console.log(`${row[0]}: যোগ হয়েছে`);
  } catch (error) {
    console.log(`${row[0]}: ${error.message}`);
  }
}
```

Output

```
SJK-2: যোগ হয়েছে
SJK-2: UNIQUE constraint failed: tours.code
KTA-0: CHECK constraint failed: price_positive
LNG-31: CHECK constraint failed: days_range
```

সমাধান ২ — আগে: `bookings(id, customer, tour_name, tour_price, date)`—Tour-এর দাম বদলালে শত সারি হালনাগাদ, এবং পুরোনো ও নতুন দাম মিশে যায়। পরে: `tours(code PK, name, price)` এবং `bookings(id, customer_id FK, tour_code FK, price_paid, date)`। লক্ষ করুন, `price_paid` ইচ্ছাকৃতভাবে Booking-এ রাখা: এটি পুনরাবৃত্তি নয়, ঐতিহাসিক সত্য—বুকিংয়ের দিনের দাম, যা পরে Tour-এর দাম বদলালেও বদলাবে না।

সমাধান ৩ — Atomicity: সব বা কিছুই না। Consistency: শেষে সব Constraint সত্য। Isolation: অন্যদের আধা-কাজ দেখা যায় না। Durability: `COMMIT`-এর পর স্থায়ী।

সমাধান ৪

email-index.mjs · JavaScript চলবে: Node.js

```
import { DatabaseSync } from 'node:sqlite';

const db = new DatabaseSync(':memory:');
db.exec('CREATE TABLE customers (id INTEGER PRIMARY KEY, email TEXT, name TEXT)');
const QUERY = 'SELECT * FROM customers WHERE email = ?';
const plan = () => db.prepare(`EXPLAIN QUERY PLAN ${QUERY}`)
  .all('a@b.c').map((row) => row.detail).join('');

console.log(plan());
db.exec('CREATE UNIQUE INDEX customers_email ON customers (email)');
console.log(plan());
```

Output

```
SCAN customers
SEARCH customers USING INDEX customers_email (email=?)
```

TravelData-র `email TEXT UNIQUE` নিজেই একটি Index তৈরি করে, তাই সেখানে আলাদা Index লাগে না।

সমাধান ৫ — একটি তালিকা আনতে ১টি Query, তারপর তালিকার প্রতিটি সারির জন্য আরও একটি করে Query (N টি)—যেমন ৫০ জন গ্রাহকের প্রত্যেকের Point আলাদা Query-তে; সমাধান JOIN বা একটি `IN (...)` Query (উদাহরণ ০৬)।

সমাধান ৬

reverse.mjs · JavaScript চলবে: Node.js

```
import { openDatabase } from '../../project/travel-data/src/db.mjs';
import { migrate } from '../../project/travel-data/src/migrate.mjs';
import { createCustomerStore } from '../../project/travel-data/src/customers.mjs';
import { createLedger } from '../../project/travel-data/src/ledger.mjs';

const db = openDatabase();
migrate(db);
const id = createCustomerStore(db).create({ name: 'Sara Islam', email: 's@e.com',
  phone: '01912345678', city: 'Bogura' });
const ledger = createLedger(db);
ledger.earn(id, 700, 'wrong tour');

function reverse(entryId) {
  const entry = db.prepare('SELECT * FROM ledger_entries WHERE id = ?')
    .get(entryId);
```

```
  return ledger.earn(entry.customer_id, -entry.points,
    `reversal of #${entryId}`, `reverse-${entryId}`);
}

reverse(1);
reverse(1);
const rows = db.prepare(
  'SELECT points, reason FROM ledger_entries ORDER BY id').all();
console.log(rows.map((r) => `${r.points} (${r.reason})`).join(' | '));
console.log(ledger.balanceOf(id).balance, ledger.audit().length);
```

Output

```
700 (wrong tour) | -700 (reversal of #1)
0 0
```

মূল সারি অপরিবর্তিত; ইতিহাসে দেখা যায় কী ঘটেছিল এবং কীভাবে সংশোধন হয়েছে। `reverse-1` Key থাকায় দ্বিতীয়বার উল্টানো হয়নি।

সমাধান ৭

004-referral.sql · SQL চলবে: Node.js

```
-- ঐচ্ছিক: পুরোনো সারি NULL থাকে, তাই কিছু ভাঙে না।
ALTER TABLE customers ADD COLUMN referred_by INTEGER
  REFERENCES customers (id) ON DELETE SET NULL;
CREATE INDEX customers_referred_by ON customers (referred_by);
```

apply.mjs · JavaScript চলবে: Node.js

```
import { cpSync, mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { fileURLToPath } from 'node:url';
import { openDatabase } from '../../../project/travel-data/src/db.mjs';
import { migrate } from '../../../project/travel-data/src/migrate.mjs';

const dir = mkdtempSync(join(tmpdir(), 'mig-'));
const project = new URL('../../../project/travel-data/migrations/',
  import.meta.url);
cpSync(fileURLToPath(project), dir, { recursive: true });
cpSync(fileURLToPath(new URL('./004-referral.sql', import.meta.url)),
  join(dir, '004-referral.sql'));

const db = openDatabase();
console.log(migrate(db, dir).join(', '));
const columns = db.prepare('PRAGMA table_info(customers)').all()
```

```
  .map((c) => c.name);
console.log(columns.includes('referred_by'));
rmSync(dir, { recursive: true, force: true });
```

Output

```
001-init.sql, 002-indexes.sql, 003-tier.sql, 004-referral.sql
true
```

সমাধান ৮

city-report.sql · SQL চলবে: Node.js

```sql
SELECT c.city, COUNT(*) AS customers, SUM(a.balance) AS points
FROM customers c
JOIN accounts a ON a.customer_id = c.id
GROUP BY c.city
ORDER BY points DESC;
```

`customers_city` Index (Migration ০০২) এই Query-র `GROUP BY`-তে সাহায্য করতে পারে; Plan দিয়ে যাচাই করুন।

সমাধান ৯ — উদাহরণ ০৪-এর ধাঁচ: `BEGIN`, প্রতিটি গ্রাহকের আগে `SAVEPOINT row`, সফল হলে `RELEASE row`, ব্যর্থ হলে `ROLLBACK TO row` ও `RELEASE row` এবং ভুলের তালিকায় যোগ, শেষে `COMMIT`। কিন্তু খেয়াল রাখুন, TravelData-র `customers.create` নিজেই `withTransaction` চালায়, এবং SQLite-এ Transaction-এর ভেতরে আরেকটি `BEGIN` চলে না—তাই Import-এর জন্য Store-এ একটি আলাদা Function লাগবে, যা Transaction খোলে না, শুধু Savepoint ব্যবহার করে। এটি একটি বাস্তব নকশার প্রশ্ন: Transaction-এর সীমানা কে ঠিক করবে—Repository, না যে Use Case তাকে ডাকে? বড় Project-এ উত্তর সাধারণত দ্বিতীয়টি (Unit of Work)।

সমাধান ১০

retry.mjs · JavaScript চলবে: Node.js

```
import { mkdtempSync, rmSync } from 'node:fs';
import { tmpdir } from 'node:os';
import { join } from 'node:path';
import { openDatabase, withTransaction }
  from '../../project/travel-data/src/db.mjs';

function withRetry(fn, { attempts = 3, onRetry = () => {} } = {}) {
  for (let i = 1; ; i += 1) {
    try {
      return fn();
    } catch (error) {
```

```
      if (!/database is locked/.test(error.message) || i >= attempts) {
        throw error;
      }
      onRetry(i);
    }
  }
}

const dir = mkdtempSync(join(tmpdir(), 'retry-'));
const a = openDatabase(join(dir, 'x.db'), { busyTimeoutMs: 0 });
const b = openDatabase(join(dir, 'x.db'), { busyTimeoutMs: 0 });
a.exec('CREATE TABLE t (n INTEGER)');

a.exec('BEGIN IMMEDIATE');
const retries = [];
const result = withRetry(() => withTransaction(b, () => {
  b.exec('INSERT INTO t VALUES (1)');
  return 'লেখা হয়েছে';
}), { onRetry: (i) => {
  retries.push(i);
  if (i === 2) {
    a.exec('COMMIT');
  }
} });
console.log(result, retries);
a.close();
b.close();
rmSync(dir, { recursive: true, force: true });
```

Output

```
লেখা হয়েছে [ 1, 2 ]
```

দুবার Lock-এ ব্যর্থ, তৃতীয়বার (অন্য সংযোগ ছেড়ে দেওয়ার পর) সফল। বাস্তবে Retry-এর মাঝে অপেক্ষা (Backoff, অধ্যায় ১৬) এবং শুধু Retry-যোগ্য Error-এ—যেমন এখানে Lock, কিন্তু Constraint কখনো নয়।

সমাধান ১১

repair.sql · SQL চলবে: Node.js

```
-- জরুরি অবস্থায়, Backup-এর পরে, একটি Transaction-এ:
UPDATE accounts SET balance = (
  SELECT COALESCE(SUM(points), 0) FROM ledger_entries e
  WHERE e.customer_id = accounts.customer_id
);
```

এটি শুধু জরুরি অবস্থায়, কারণ: Audit-এ অমিল মানে কোথাও একটি Bug বা নিয়মবহির্ভূত লেখা আছে—Repair সেই কারণ লুকিয়ে ফেলে; বড় Table-এ এটি দীর্ঘ সময় Lock ধরে রাখে; এবং যদি Ledger-ই ভুল হয়, Repair ভুলটিকে Balance-এও ছড়িয়ে দেয়। আগে কারণ খুঁজুন, Backup নিন, তারপর Repair, এবং কী বদলাল তা Log করুন।

সমাধান ১২ — Table: `leads(id TEXT PRIMARY KEY, name, phone CHECK(...), destination, travelers CHECK (travelers BETWEEN 1 AND 50), status CHECK (status IN (...)), assignee, version INTEGER NOT NULL DEFAULT 1, created_at)` । Index: `(assignee, id)` ও `(status, id)` —তালিকা ও Filter-এর Keyset Pagination-এর জন্য। Idempotency: `idempotency_keys(user_id, key, fingerprint, lead_id, created_at, PRIMARY KEY (user_id, key))` , Lead তৈরি ও Key সংরক্ষণ একই Transaction-এ। Version: `UPDATE leads SET ..., version = version + 1 WHERE id = ? AND version = ?` এবং `changes === 0` হলে 412 (অধ্যায় ২৫-এর সমাধান ১২)। ID: `AUTOINCREMENT` থেকে, গণনা থেকে নয়। সবকিছু `001-init.sql` -এ, এবং Service-এর একটি লাইনও বদলায় না।

২৬.১৭ MCQ: নিজেকে যাচাই করুন

প্রশ্ন ১. Normalization-এর প্রধান লক্ষ্য?

ক) Query দ্রুত করা

খ) প্রতিটি তথ্য একবার রেখে পুনরাবৃত্তি ও অসামঞ্জস্য দূর করা

গ) Table-এর সংখ্যা কমানো

ঘ) Index তৈরি

প্রশ্ন ২. SQLite-এ Foreign Key যাচাইয়ের জন্য কী লাগে?

ক) `PRAGMA foreign_keys = ON` , প্রতিটি সংযোগে

খ) কিছুই না, সব সময় চালু

গ) একটি Index

ঘ) `UNIQUE`

প্রশ্ন ৩. Transaction-এর ভেতরে দ্বিতীয় লেখা ব্যর্থ হলে `ROLLBACK` -এর পর?

ক) প্রথম লেখা থেকে যায়

খ) Database ক্ষতিগ্রস্ত

গ) দুটি লেখাই বাতিল

ঘ) দ্বিতীয়টি আবার চেষ্টা হয়

প্রশ্ন ৪. Lost Update কেন ঘটে?

ক) Index নেই

খ) Backup নেই

গ) Constraint ভুল

ঘ) দুটি সংযোগ একই মান পড়ে, আলাদা হিসাব করে লেখে

প্রশ্ন ৫. `EXPLAIN QUERY PLAN`-এ `SCAN ledger_entries` মানে?

ক) পুরো Table পড়া হচ্ছে

খ) Index ব্যবহার হচ্ছে

গ) Query ব্যর্থ

ঘ) Cache থেকে

প্রশ্ন ৬. N+1 সমস্যার সাধারণ সমাধান?

ক) আরও Index

খ) JOIN বা Eager Loading-এ একসঙ্গে আনা

গ) Cache বন্ধ

ঘ) Transaction

প্রশ্ন ৭. চলে যাওয়া Migration-এ পরিবর্তন লাগলে?

ক) File সম্পাদনা

খ) Database মুছে নতুন করে

গ) হাতে `ALTER TABLE`

ঘ) একটি নতুন Migration লেখা

প্রশ্ন ৮. গভীর পাতায় `OFFSET`-এর বদলে কী ভালো?

ক) `WHERE id > ? ORDER BY id LIMIT ?` (Keyset)

খ) বড় `LIMIT`

গ) সব সারি একসঙ্গে

ঘ) `ORDER BY RANDOM()`

প্রশ্ন ৯. Idempotency-র সবচেয়ে নির্ভরযোগ্য Database-স্তরের নিশ্চয়তা?

ক) একটি `if`

খ) Key-তে `UNIQUE` Constraint

গ) Cache

ঘ) Index ছাড়া Table

> **প্রশ্ন ১০.** Backup কখন নির্ভরযোগ্য বলা যায়?
>
> ক) File তৈরি হলে
>
> খ) প্রতিদিন হলে
>
> গ) Restore ও যাচাই করে দেখার পর
>
> ঘ) বড় হলে

২৬.১৮ MCQ-এর উত্তর ও ব্যাখ্যা

দ্রুত উত্তরমালা: ১–খ · ২–ক · ৩–গ · ৪–ঘ · ৫–ক · ৬–খ · ৭–ঘ · ৮–ক · ৯–খ · ১০–গ

প্রশ্ন	উত্তর	ব্যাখ্যা
১	খ	উদাহরণ ০১-এর একই গ্রাহকের দুটি ফোন নম্বর।
২	ক	SQLite-এ Default বন্ধ; `openDatabase` চালু করে।
৩	গ	Atomicity; উদাহরণ ০৩-এ প্রমাণিত।
৪	ঘ	দ্বিতীয় লেখা প্রথমটিকে মুছে দেয়।
৫	ক	Index থাকলে SEARCH দেখা যায়।
৬	খ	৫১টির বদলে ১টি Query।
৭	ঘ	Checksum পরিবর্তন ধরে থামিয়ে দেয়।
৮	ক	Index দিয়ে সরাসরি পরের পাতায়।
৯	খ	একসঙ্গে আসা অনুরোধেও কাজ করে।
১০	গ	`backupTo` তাই খুলে যাচাই করে।

২৬.১৯ অধ্যায়ের সারসংক্ষেপ

- Normalization: প্রতিটি তথ্য একবার, তার মালিক Table-এ; Denormalization শুধু সচেতনভাবে, Audit-সহ।
- Constraint (PK, FK, UNIQUE, CHECK, NOT NULL) Database-কে নিয়মের পাহারাদার করে; নাম দিন।
- Transaction: ACID; একসঙ্গে বদলানো সব লেখা একটিতে; Savepoint দিয়ে আংশিক ফেরা।
- Isolation ও Lock; Lost Update প্রতিরোধে Database-এর ভেতরে হিসাব, Lock বা Version।
- Index ও Query Plan; N+1-এর বদলে JOIN; Keyset Pagination।
- Migration: ক্রমিক, Transaction-এ, Checksum-সহ; পুরোনোটি কখনো বদলাবেন না।
- Backup যাচাই না হলে Backup নয়।

- ORM সুবিধাজনক, কিন্তু SQL ও Plan জানা জরুরি; SQLite ও PostgreSQL-এর পার্থক্য জেনে Adapter।

পরের অধ্যায়ে এই বইয়ের যাত্রা আরও এগোবে; এ পর্যন্ত আপনি Browser থেকে Server, Test থেকে Database পর্যন্ত একটি পেশাদার JavaScript Application-এর প্রতিটি স্তর তৈরি করেছেন।

২৬.২০ গুরুত্বপূর্ণ পরিভাষা

পরিভাষা	অর্থ
Primary Key / Foreign Key	সারির অনন্য পরিচয় / অন্য Table-এর সারির উল্লেখ।
Constraint	Database-এর ভেতরের নিয়ম।
Normalization	পুনরাবৃত্তি দূর করে তথ্য সাজানো।
Transaction (ACID)	অবিভাজ্য কাজের একক: Atomicity, Consistency, Isolation, Durability।
Savepoint	Transaction-এর ভেতরে ফেরার বিন্দু।
Lost Update	একটি হালনাগাদ অন্যটিকে মুছে দেওয়া।
Index	দ্রুত খোঁজার সাজানো কাঠামো।
Query Plan	Database Query কীভাবে চালাবে তার বর্ণনা।
N+1	তালিকার প্রতিটি সারির জন্য আলাদা Query।
Migration	Schema পরিবর্তনের ক্রমিক File।
ORM	Table-কে Object হিসেবে ব্যবহারের Library।
Append-only Ledger	শুধু যোগ হয় এমন লেনদেনের তালিকা।

২৬.২১ আরও পড়ুন

- **Node.js — SQLite:** `https://nodejs.org/api/sqlite.html` — `node:sqlite`-এর API।
- **SQLite — Query Planning:** `https://www.sqlite.org/queryplanner.html` — Index ও Plan কীভাবে কাজ করে।
- **PostgreSQL — Transaction Isolation:** `https://www.postgresql.org/docs/current/transaction-iso.html` — Isolation Level।
- **PostgreSQL — Using EXPLAIN:** `https://www.postgresql.org/docs/current/using-explain.html` — Query Plan পড়া।
- **Prisma — Documentation:** `https://www.prisma.io/docs` — ORM, Migration ও Client।